

Chapter 4

Linked Lists

The representation of simple data structure using an **array** and a **sequential mapping** has the **property**:

- ◆ Successive nodes of the data object are stored at fixed distance apart.
 - ◆ (physical order $\leftrightarrow$ logical order)
- ◆ This makes it easy to access an arbitrary node in $O(1)$.

Disadvantage of sequential mapping:

It makes insertion and deletion of arbitrary elements expensive.

For example:

**Insert “GAT” into or delete “LAT” from
(BAT, CAT, EAT, FAT, HAT, JAT, LAT, MAT, OAT, PAT, RAT,
SAT, TAT, VAT, WAT)**

need data movement.

Solution---linked representation:

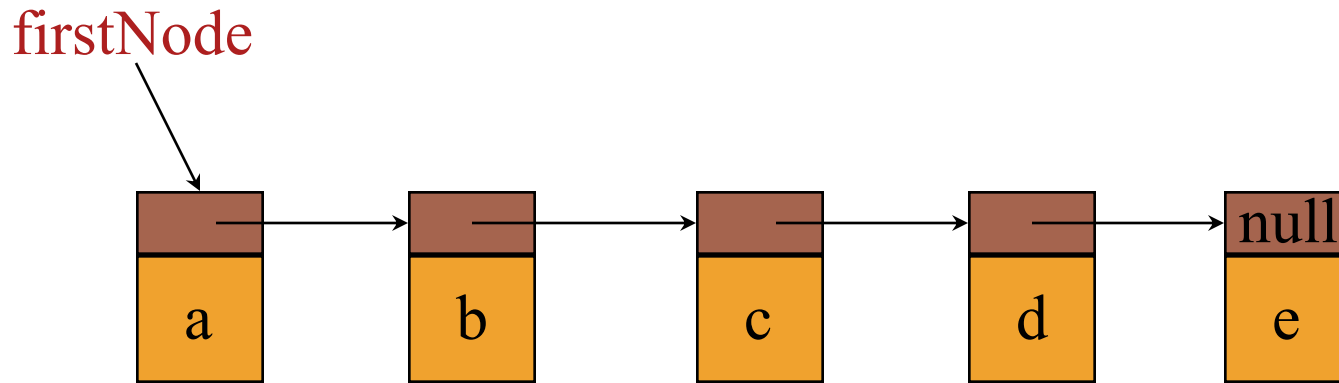
**items of a list may be placed anywhere in the
memory.**

**Associated with each item is a point (link) to the next
item.**

The above structures are called **singly/singular linked lists** or **chains** in which each node has exactly **one** pointer field.

- list elements are stored, in memory, in an **arbitrary order**
- **explicit information** (called a link) is used to go from one element to the next

Chain



- A chain is a linked list in which each node represents one element.
- There is a link or pointer from one element to the next.
- The last node has a **null** pointer.

Memory Layout

Layout of $L = (a,b,c,d,e)$ using an array representation.

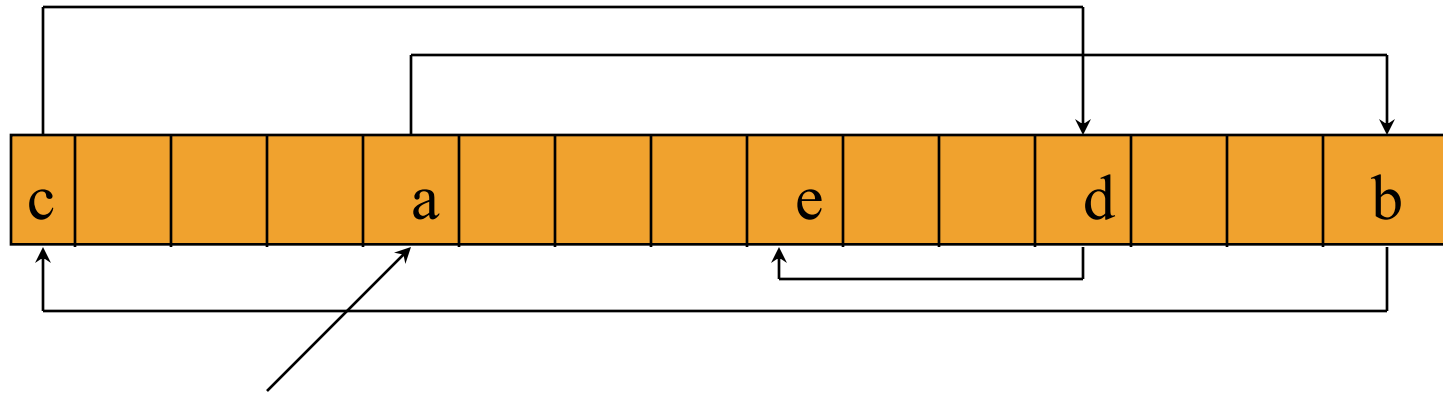


A linked representation uses an arbitrary layout.





Linked Representation

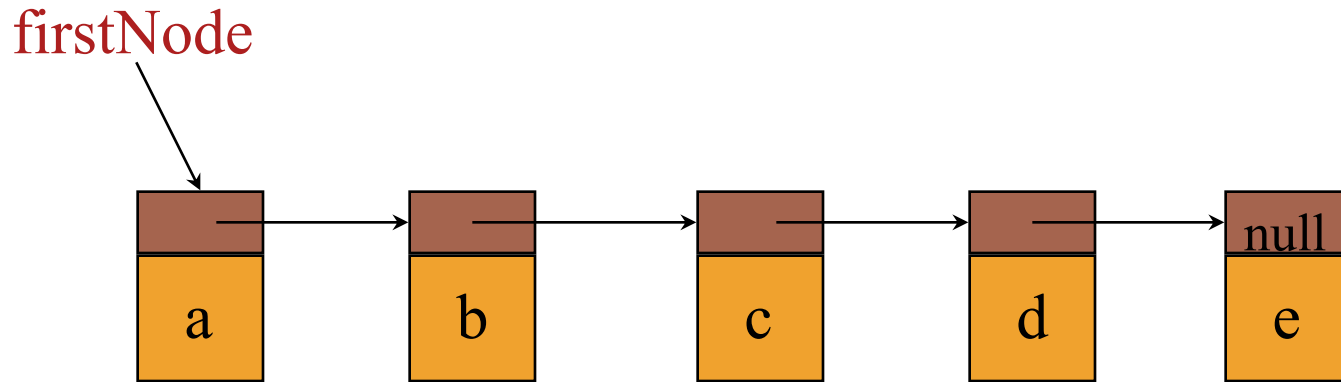


firstNode

pointer (or link) in **e** is **null**

use a variable **firstNode** to get to the
first element **a**

Normal Way To Draw A Linked List

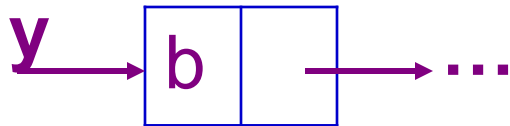


link or pointer field of node

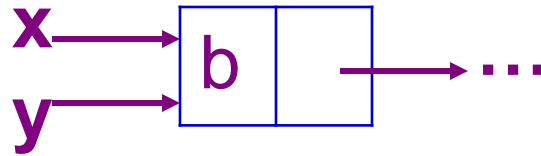
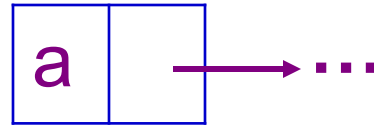


data field of node

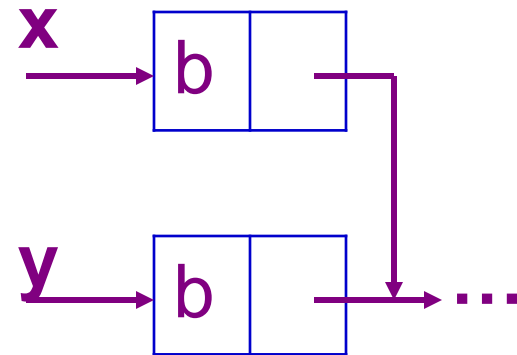
Pointer manipulation in C++:



(a)



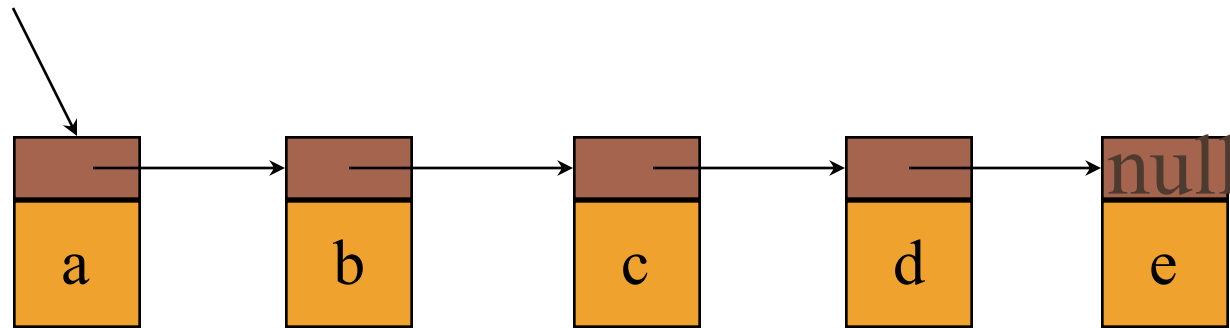
(b) $x=y$



(c) $*x=*y$

get(0)

firstNode



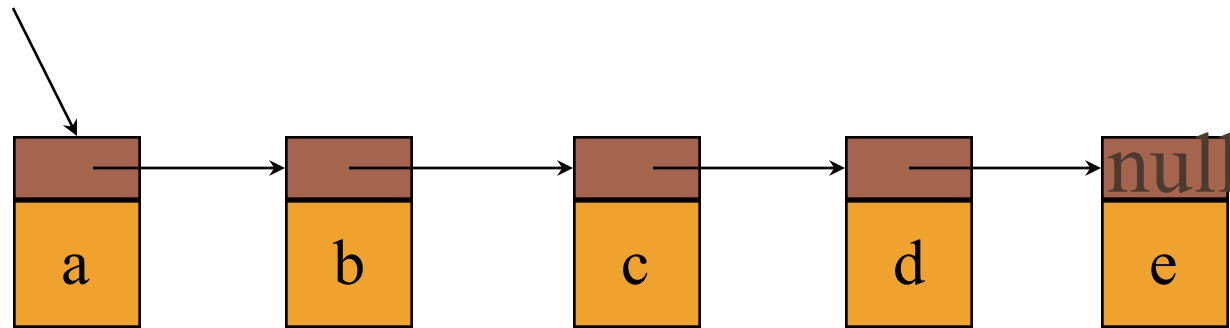
```
checkIndex(0);
```

```
desiredNode = firstNode; // gets you to first node
```

```
return desiredNode→element;
```

get(1)

firstNode



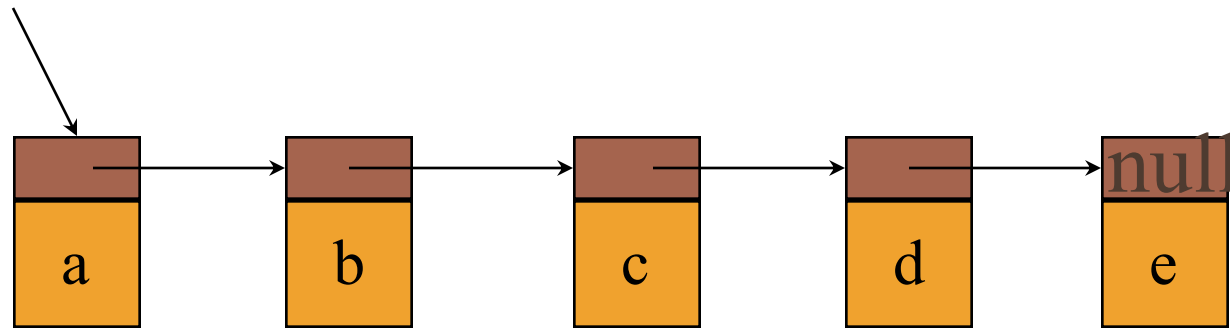
checkIndex(1);

desiredNode = firstNode → next; // gets you to second node

return desiredNode → element;

get(2)

firstNode



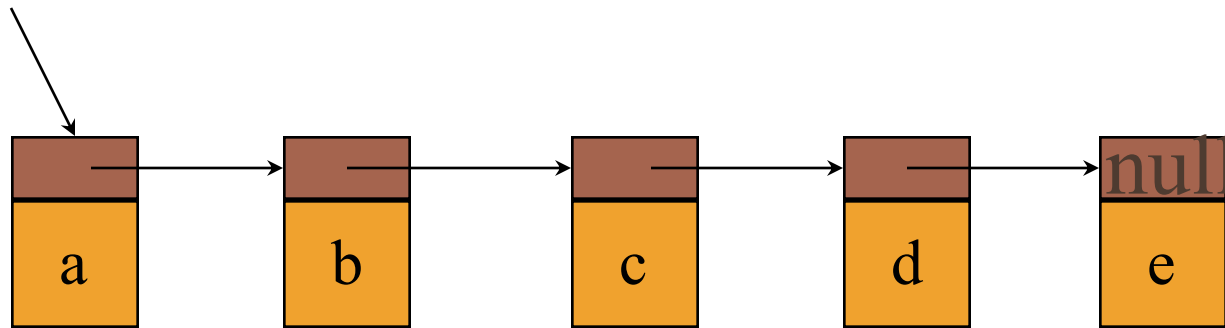
```
checkIndex(2);
```

```
desiredNode = firstNode → next → next; // gets you to third  
node
```

```
return desiredNode → element;
```

get(5)

firstNode



checkIndex(5); // throws exception

desiredNode =

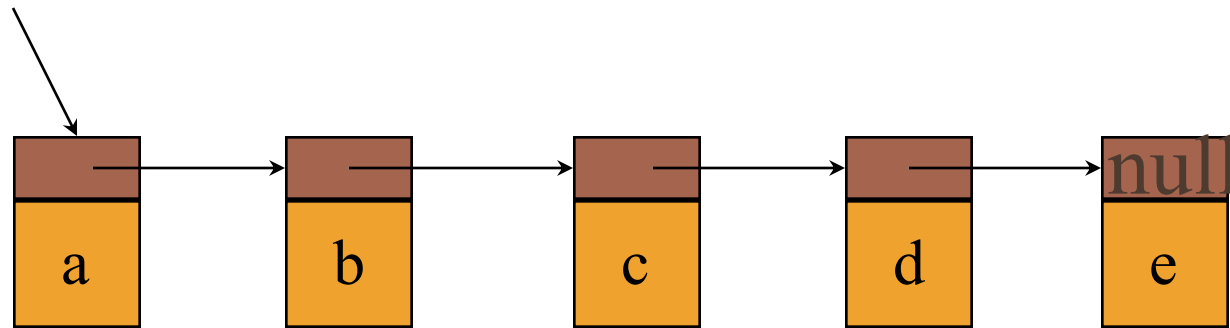
firstNode → next → next → next → next → next;

// desiredNode = null

return desiredNode → element; // null → element

NullPointerException

firstNode



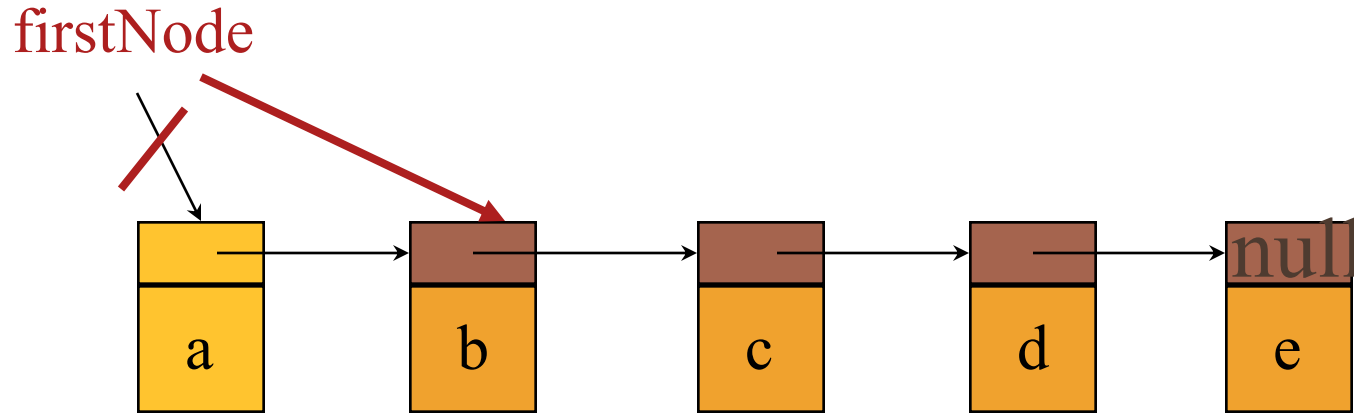
desiredNode =

```
firstNode → next → next → next → next → next;
```

// gets the computer mad

// you get a NullPointerException

Remove An Element

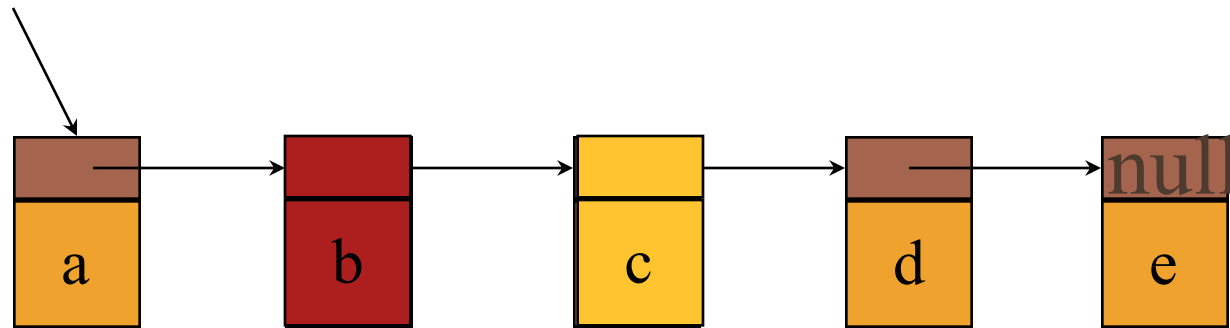


`remove(0)`

`firstNode = firstNode → next;`

remove(2)

firstNode

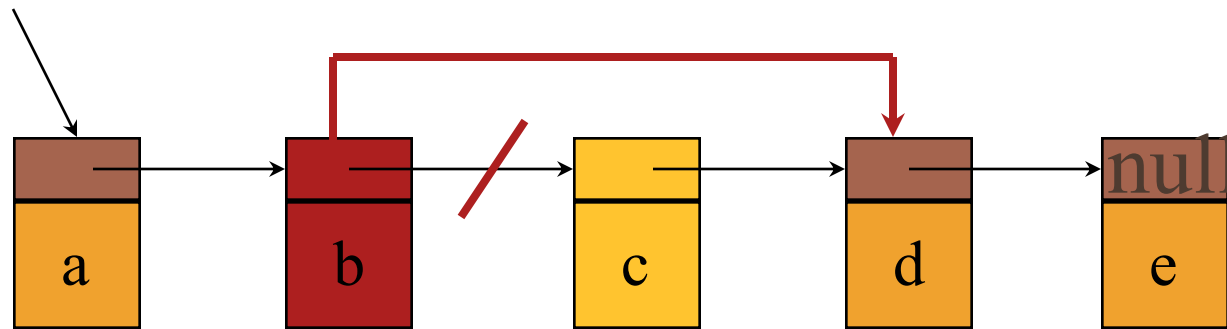


first get to node just before node to be removed

$\text{beforeNode} = \text{firstNode} \rightarrow \text{next};$

remove(2)

firstNode

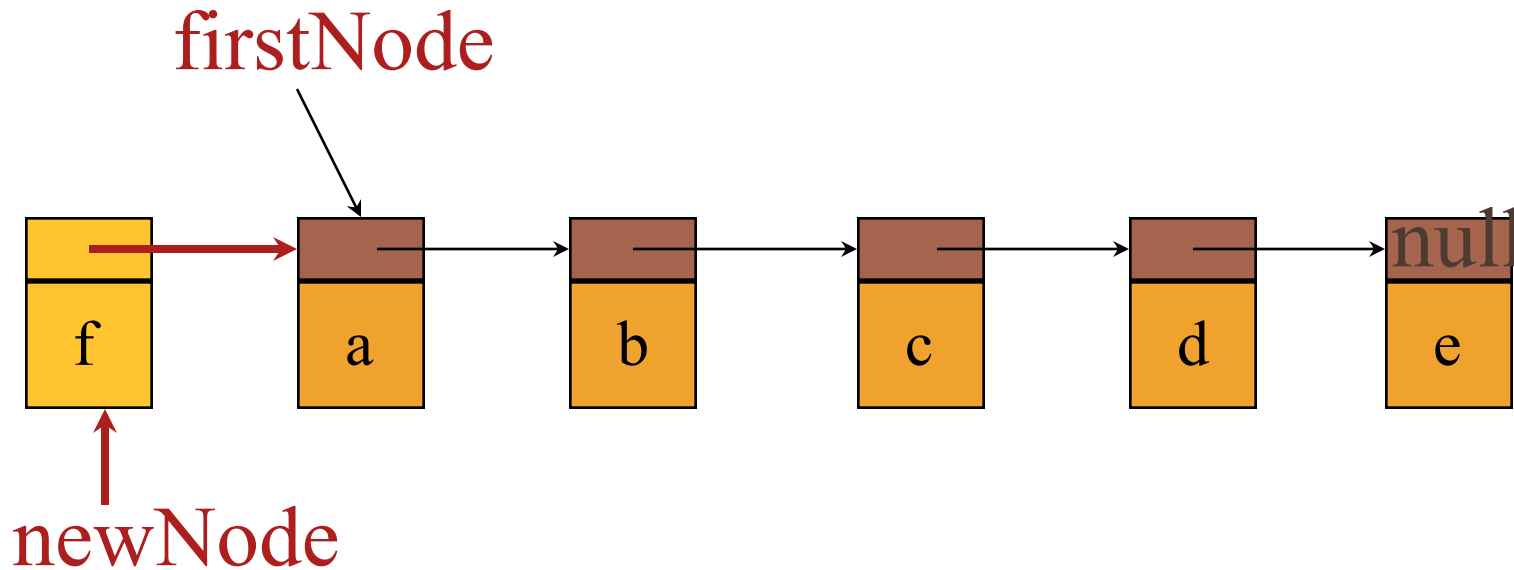


beforeNode

now change pointer in beforeNode

$\text{beforeNode} \rightarrow \text{next} = \text{beforeNode} \rightarrow \text{next} \rightarrow \text{next};$

add(0,'f')

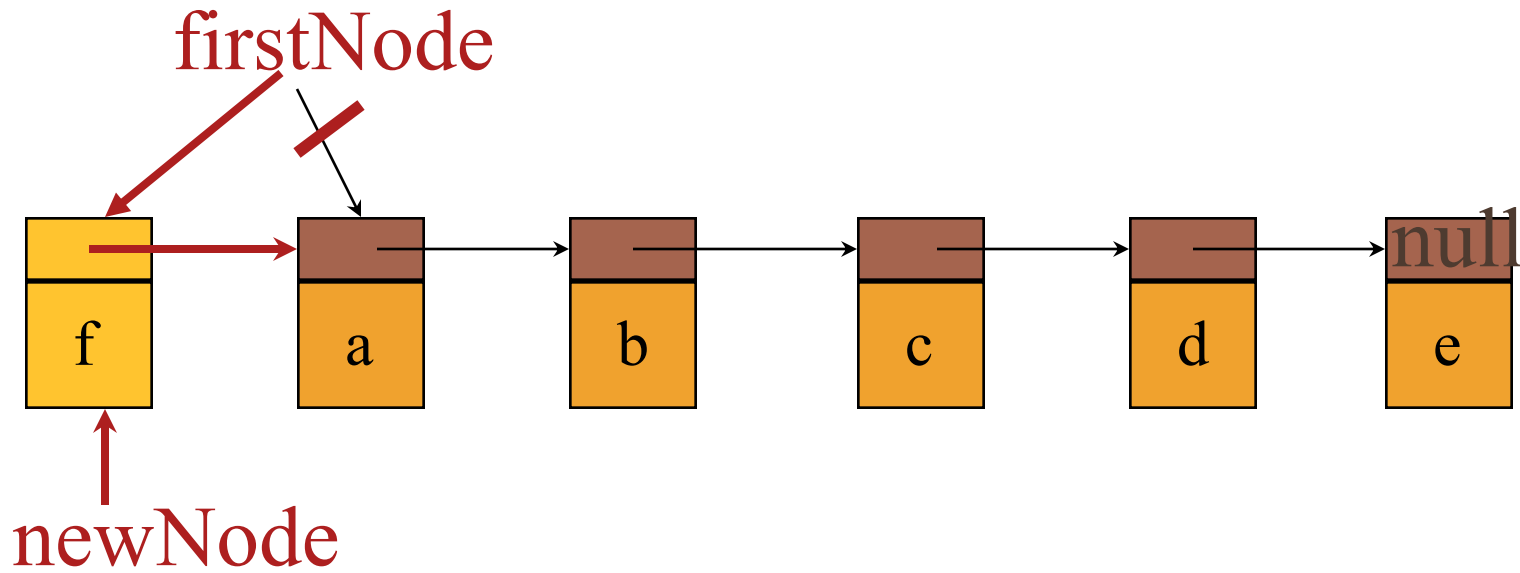


Step 1: get a node, set its data and link fields

ChainNode newNode =

new ChainNode(new Character('f'), firstNode);

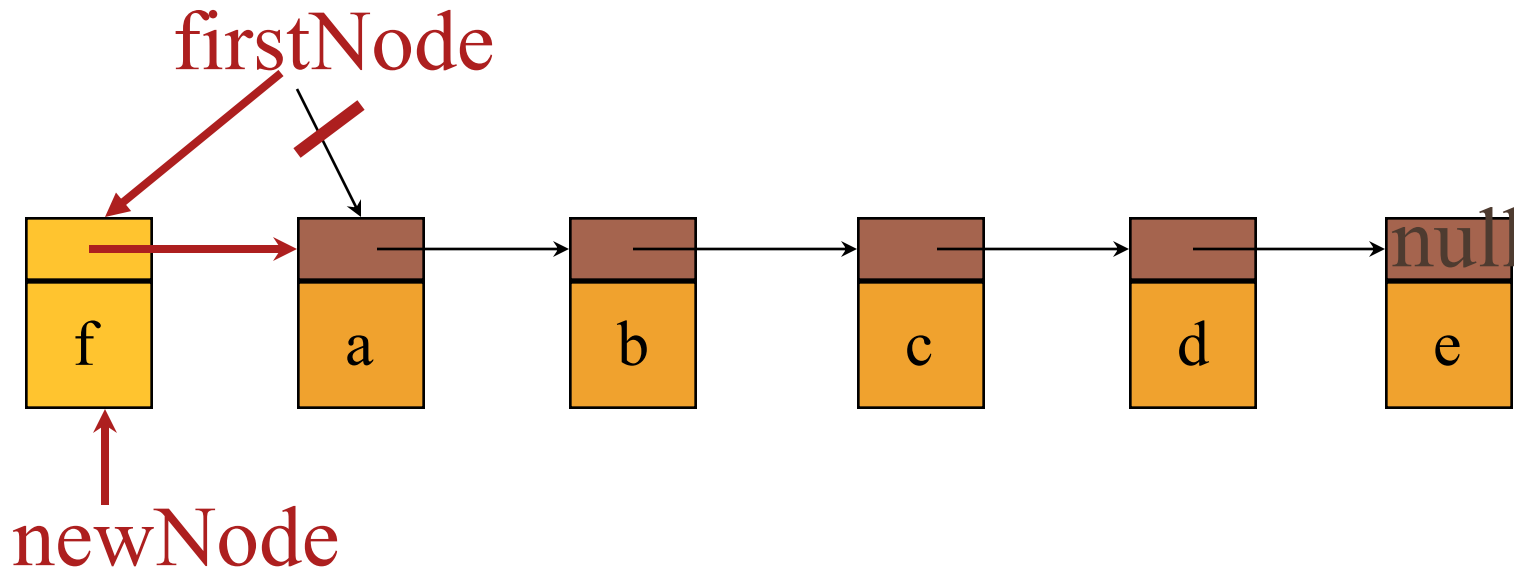
add(0,'f')



Step 2: update **firstNode**

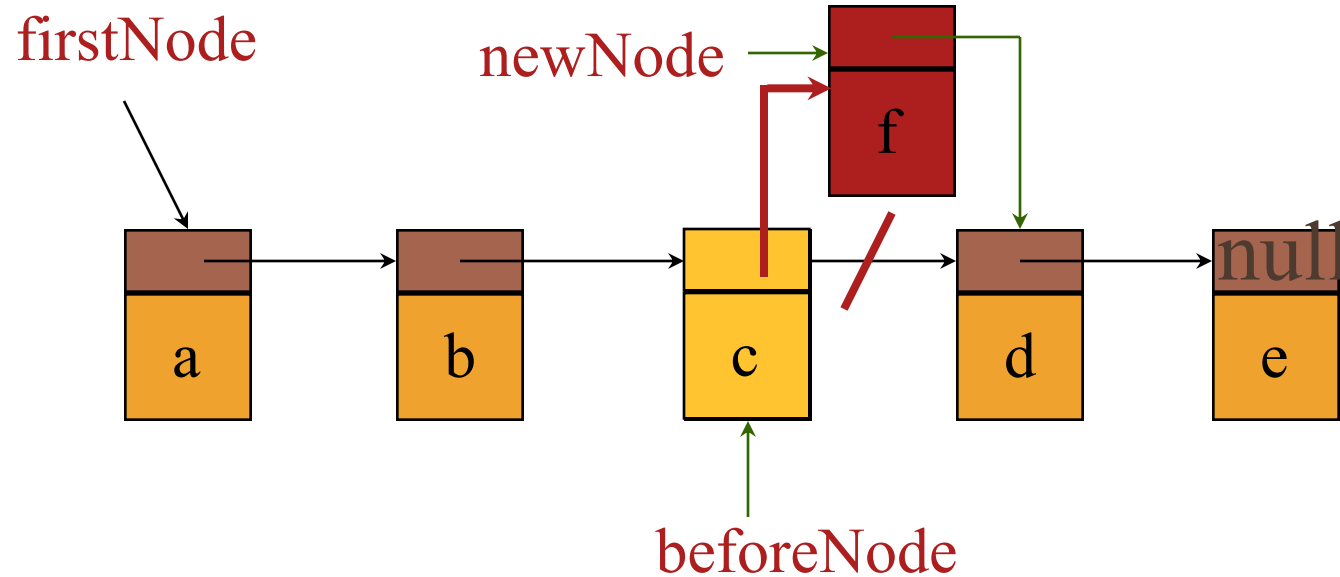
firstNode = newNode;

One-Step add(0,'f')



```
firstNode = new ChainNode(  
    new Character('f'), firstNode);
```

add(3,'f')



- first find node whose index is 2
- next create a node and set its data and link fields

```
ChainNode newNode = new ChainNode(new Character('f'),  
                                   beforeNode→next);
```

- finally link beforeNode to newNode

```
beforeNode→next = newNode;
```

4.2 Representing Chains in C++

Assume a chain node is defined as:

```
class ChainNode {  
private:  
    int data;  
    ChainNode *link;  
};
```

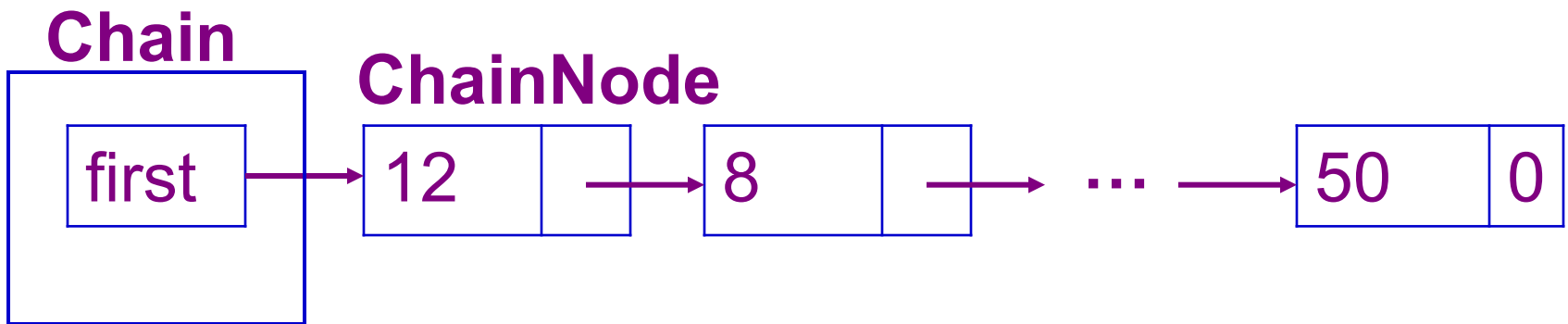
ChainNode *f;

f→data

will cause a compiler error because a private data member cannot be accessed from outside of the object.

Definition: a data object of Type A **HAS-A** data object of Type B if A conceptually contains B or B is a part of A.

A composite of two classes: ChainNode and Chain.
Chain HAS-A ChainNode.



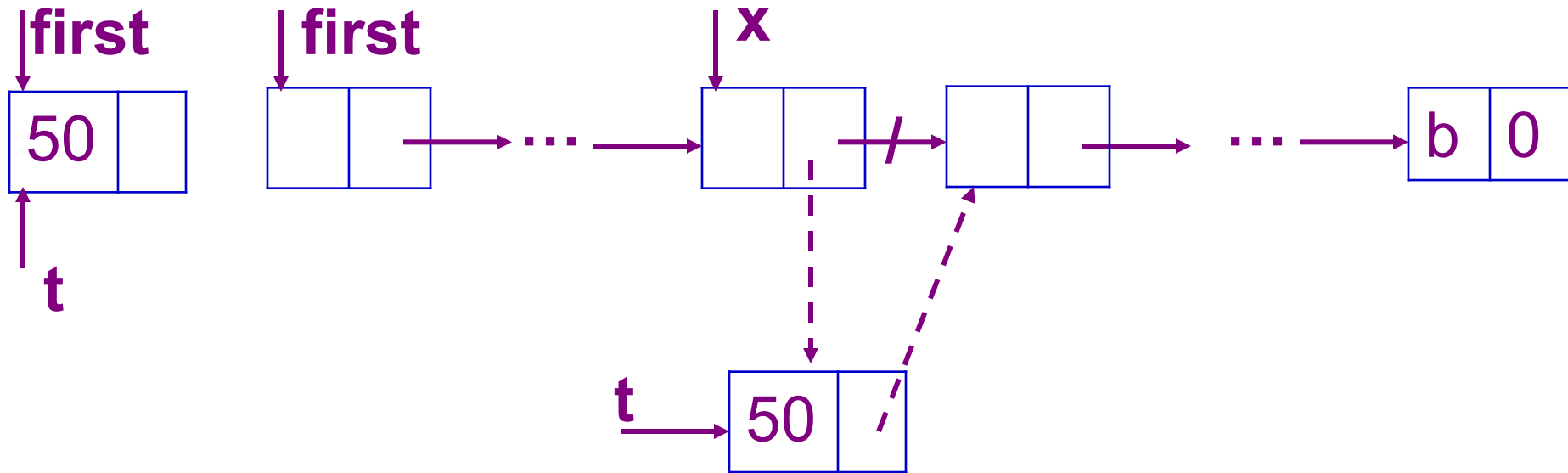
```

class Chain; // forward declaration
class ChainNode {
friend class Chain; // to make functions of Chain be able to
                      // access private data members of ChainNode
Public:
    ChainNode(int element = 0, ChainNode* next = 0)
        {data = element; link = next;}
private:
    int data;
    ChainNode *link;
};
class Chain {
public:
    // Chain manipulation operations
    ...
private:
    ChainNode *first;
};

```

Chain manipulation:

Example 4.3 insert a node with data field 50 following the node x.



```
void Chain::Insert50 (ChainNode *x)
{
    if (first)
        // insert after x
        x→link = new ChainNode(50, x→link);
    else
        // insert into empty chain
        first = new ChainNode(50);
}
```

Exercises: P183-1,2

4.3 The Template Class Chain

We shall enhance the chain class of the previous section to make it more reusable.

4.3.1 Implementing Chains with Templates

```
template <class T> class Chain; // forward declaration
```

```
template <class T>  
class ChainNode {  
friend class Chain<T>;
```

```
public:
```

```
    ChainNode(T element, ChainNode* next = 0)  
        { data = element; link = next;}
```

```
private:
```

```
    T data;  
    ChainNode *link;
```

```
};
```

```
template <class T>
class Chain {
public:
    Chain() { first=0;}; // constructor initializing first to 0
    // Chain manipulation operations
    ...
private:
    ChainNode<T> *first;
};
```

A empty chain of integers intchain would be defined as:

```
Chain<int> intchain ;
```

4.3.2 Chain Iterators

A **container** class is a class that represents a data structure that contains or stores a number of data objects.

An **iterator** is an object that is used to access the elements of a container class one by one.

Why we need an iterator?

Consider the following operations that might be performed on a container class C, all of whose elements are integers:

(1) Output all integers in C.

(2) Obtain the sum, maximum, minimum, mean, median of all integers in C.

(3) Obtain the integer x from C such that $f(x)$ is maximum.

.....

These operations have to be implemented as **member functions** of **C** to access its **private** data members.

Consider the container class **Chain<T>**, there are, however, some drawbacks to this:

- (1) All operations of **Chain<T>** should preferably be independent of the type of object to which **T** is initialized. However, operations that make sense for one instantiation of **T** may not for another instantiation.
- (2) The number of operations of **Chain<T>** can become too large.

Consider the container class `Chain<T>`, there are, however, some drawbacks to this (cont'd):

(3) Even if it is acceptable to add member functions, the user would have to learn how to sequence through the container class.

These suggest that container class be equipped with **iterators that provide **systematic access the elements of the object**.**

User can employ these iterators to implement their own functions depending upon the particular application.

Typically, an iterator is implemented as a **nested class of the container class.**

A forward Iterator for Chain

A forward Iterator class for Chain may be implemented as in the next slides, and it is required that ChainIterator be a public nested member class of Chain.

```
class ChainIterator {
```

```
public:
```

```
    // typedefs required by C++ omitted
```

```
    // constructor
```

```
    ChainIterator(ChainNode<T>* startNode = 0)
```

```
    { current = startNode; }
```

```
    // dereferencing operators
```

```
    T& operator *() const { return current→data;}
```

```
    T* operator →() const { return &current→data;}
```

```
// increment
```

```
ChainIterator& operator ++() // preincrement
```

```
{  
    current = current→link;  
    return *this;  
}
```

```
ChainIterator& operator ++(int) // postincrement
```

```
{  
    ChainIterator old = *this;  
    current = current→link;  
    return old;  
}
```

// equality testing

```
bool operator !=(const ChainIterator right) const  
    { return current != right.current; }
```

```
bool operator == (const ChainIterator right) const  
    { return current == right.current; }
```

private:

```
    ChainNode<T>* current;
```

```
};
```

Additionally, we add the following public member functions to **Chain**:

```
ChainIterator begin() {return ChainIterator(first);}
```

```
ChainIterator end() {return ChainIterator(0);}
```

We may initialize an iterator object **yi** to the start of a chain of integers **y** using the statement:

```
Chain<int>::ChainIterator yi = y.begin();
```

And we may sum the elements in **y** using the statement:

```
sum = accumulate(y.begin(), y.end(), 0);
```

```
// note sum does not require access to private members
```

Write an algorithm to print all data of a Chain.

Write an algorithm to print all data of a Chain using the iterator mechanism.

Exercises: P194- 4

4.3.3 Chain Operations

Operations provided in a reusable class should be enough but not too many.

Normally, include: constructor, destructor, operator=, operator==, operator>>, operator<<, etc.

A chain class should provide functions to **insert and **delete** elements.**

Another useful function is reverse that does an “in-place” reversal of the elements in a chain.

To be efficient, we add a private member **last** to **Chain<T>**, which points to the last node in the chain.

Write an algorithm to find the last node of a Chain

InsertBack

```
template <class T>
void Chain<T>::InsertBack(const T& e)
{
    if (first) { // nonempty chain
        last→link = new ChainNode<T>(e);
        last = last→link;
    }
    else first = last = new ChainNode<T>(e);
}
```

The complexity: $O(1)$.

Concatenate

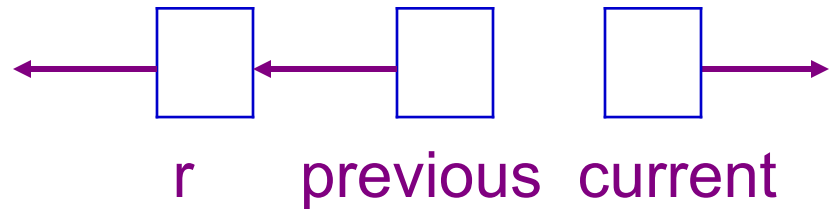
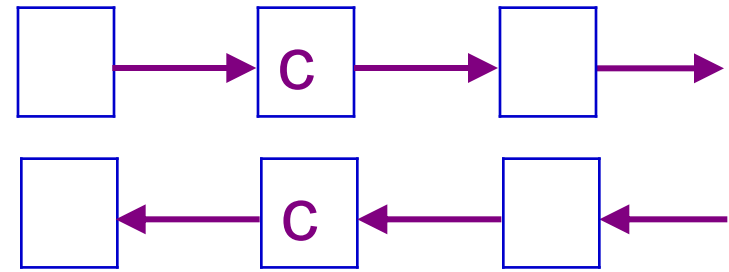
```
template <class T>
void Chain<T>::Concatenate(Chain<T>& b)
{ // b is concatenated to the end of *this
    if (first)
        { last→link = b.first; last = b.last; }
    else
        { first = b.first; last = b.last; };
    b.first = b.last = 0;
}
```

The complexity: $O(1)$.

Reverse

template <class T>

void Chain<T>::Reverse()

$$\{$$

$$\}$$

Reverse

```
template <class T>
```

```
void Chain<T>::Reverse()
```

```
{ // make  $(a_1, \dots, a_n)$  becomes  $(a_n, \dots, a_1)$ .
```

```
    ChainNode<T> *current = first, *previous = 0;
```

```
    while (current) {
```

```
        ChainNode<T> *r = previous; // r trails previous
```

```
        previous = current;
```

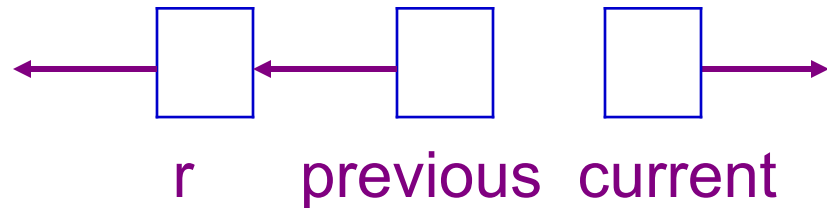
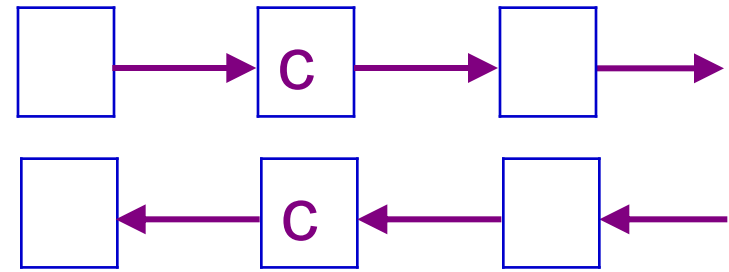
```
        current = current→link;
```

```
        previous→link = r;
```

```
    }
```

```
    first = previous;
```

```
}
```



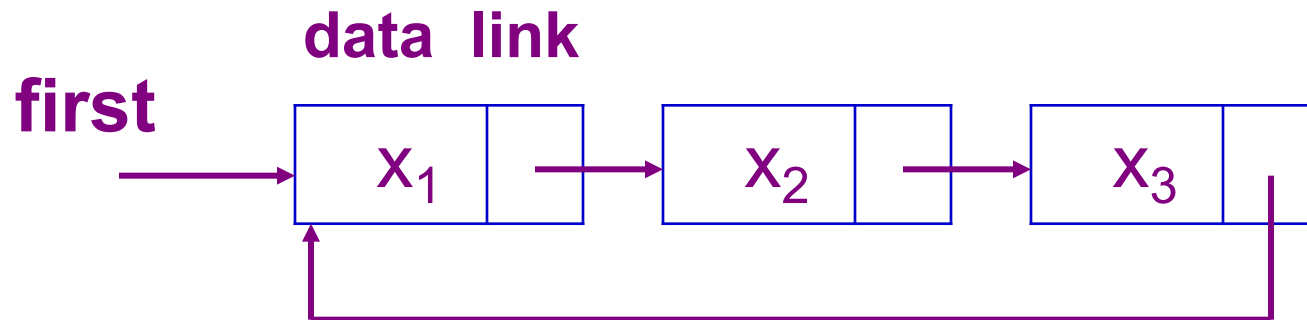
For a chain with $m \geq 1$ nodes, the computing time of Reverse is $O(m)$.

Write an algorithm to construct a Chain from an Array.

Exercises: P184- 5,6

4.4 Circular Lists

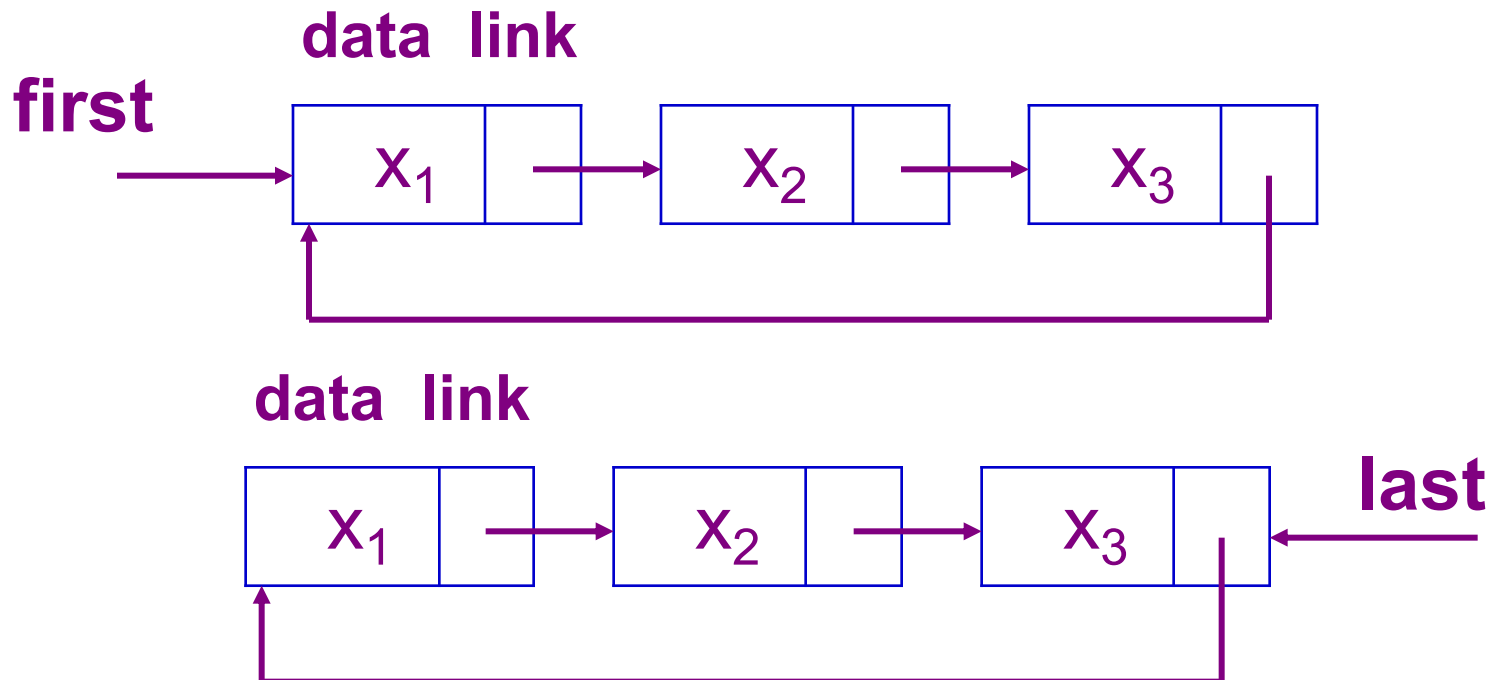
A circular list can be obtained by making the **link** field of the last node points to the first node of a chain.



Consider inserting a new node at the front

We need to change the link field of the node containing x_3 .

It is more convenient if the access pointer points to the **last** rather than the first.



Now we can insert at the front in $O(1)$:

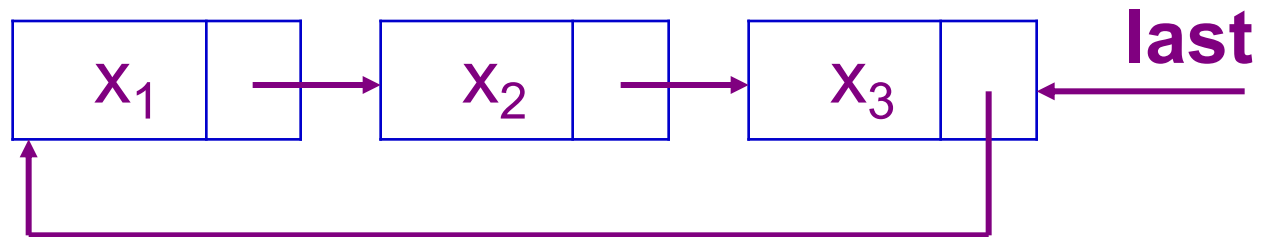
```
template <class T>
void CircularList<T>::InsertFront(const T& e)
{ // insert the element e at the “front” of the circular list *this,
  // where last points to the last node in the list.
  ChainNode<T>* newNode = new ChainNode<T>(e);
  if (last) { // nonempty list
    newNode→link =
                                last→link;
    last→link =
                                newNode;
  }
  else { last = newNode; newNode→link = newNode; }
}
```

To insert at the **back**,

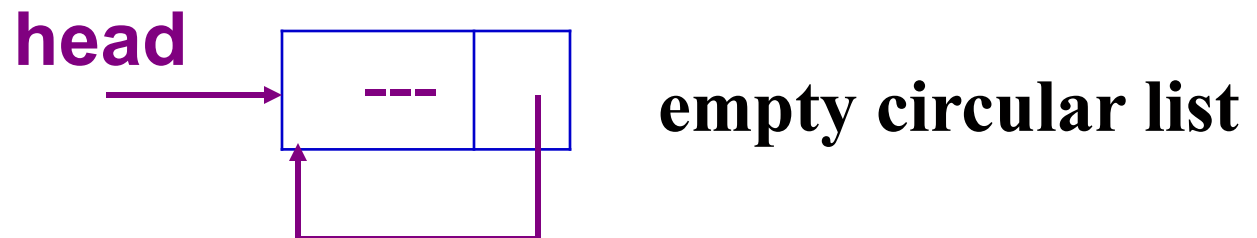
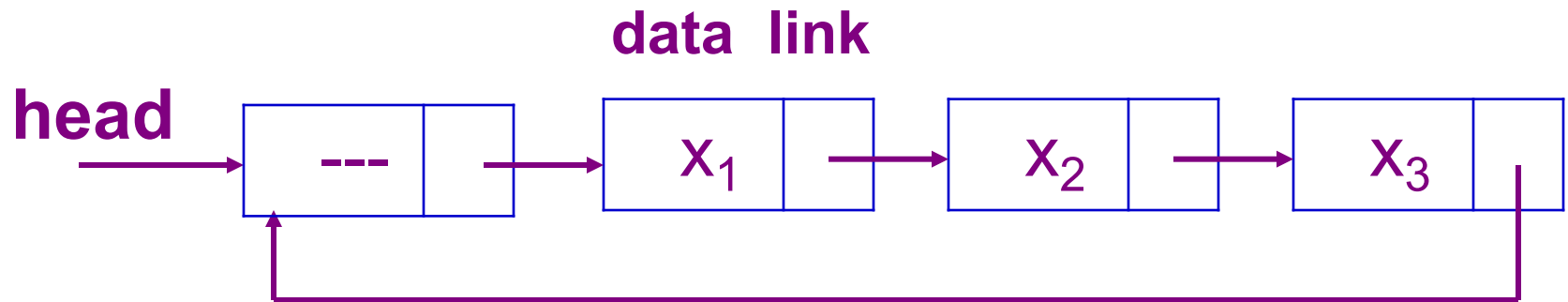
we only need to add the statement

last = newNode;

to the if clause of **InsertFront**, the complexity is still **O(1)**.



To avoid handling empty list as a special case
introduce a dummy **head** node:.



4.5 Available Space lists

- the time of destructors for chains and circular lists is **linear** in the length of the chain or list.
- it may be reduced to $O(1)$ if we maintain our own chain of free nodes.
- the available space list is pointed by **av**.
- **av** be a static class member of **CircularList<T>** of type **ChainNode<T> ***, initially, **av** = 0.
- only when the **av** list is empty do we need use **new**.

We shall now use `CircularList<T>::GetNode` instead of using `new`:

```
template <class T>  
ChainNode<T>* CircularList<T>::GetNode( )  
  
{ //provide a node for use  
    ChainNode<T> * x;  
    if (av) { x = av; av = av→link;}  
    else x = new ChainNode<T>;  
    return x;  
}
```

And we use **CircularList<T>::RetNode** instead of using **delete**:

```
template <class T>
void CircularList<T>::RetNode(ChainNode<T>*& x)
{ // free the node pointed to by x
  x→link = av;
  av = x;
  x = 0;
}
```

A circular list may be destructed in $O(1)$:

```
template <class T>
void CircularList<T>::~~CircularList()
{ // delete the circular list.
    if (last) {
        ChainNode <T> * first = last→link;
        last→link = av; // (1)
        av = first; // (2)
        last = 0;
    }
}
```

As shown in the next slide:

```

template <class T>
void CircularList<T>::~~CircularList()
{ // delete the circular list.
    if (last) { ChainNode <T> * first = last->link;
        last->link = av; // (1)
        av = first; // (2)
        last = 0;
    }
}

```



A chain may be deleted in $O(1)$ if we know its **first** and **last** nodes:

```
template <class T>
Chain<T>::~~Chain()
{ // delete the chain
    if (first) {
        last→link = av;
        av = first;
        first = 0;
    }
}
```